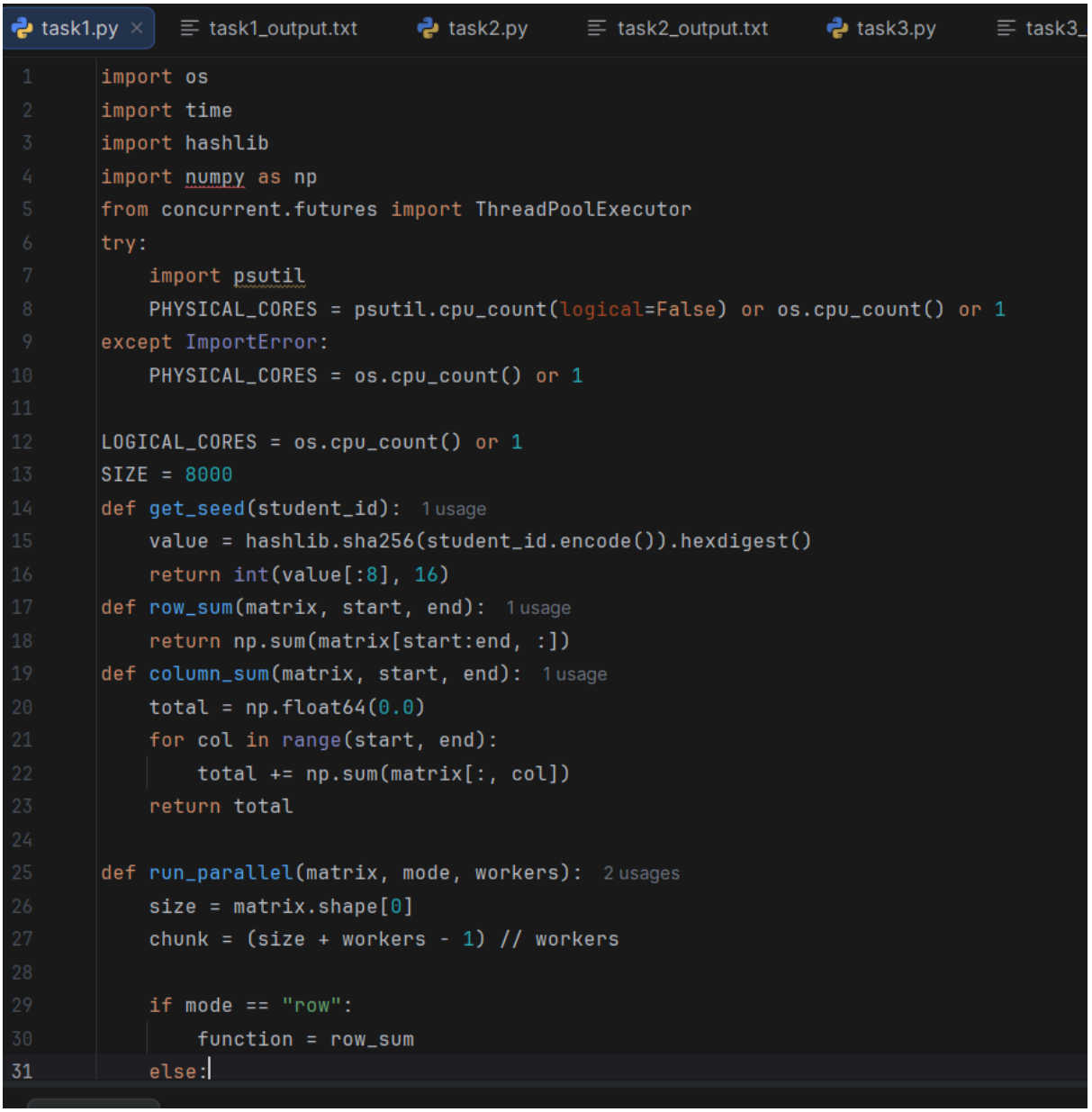
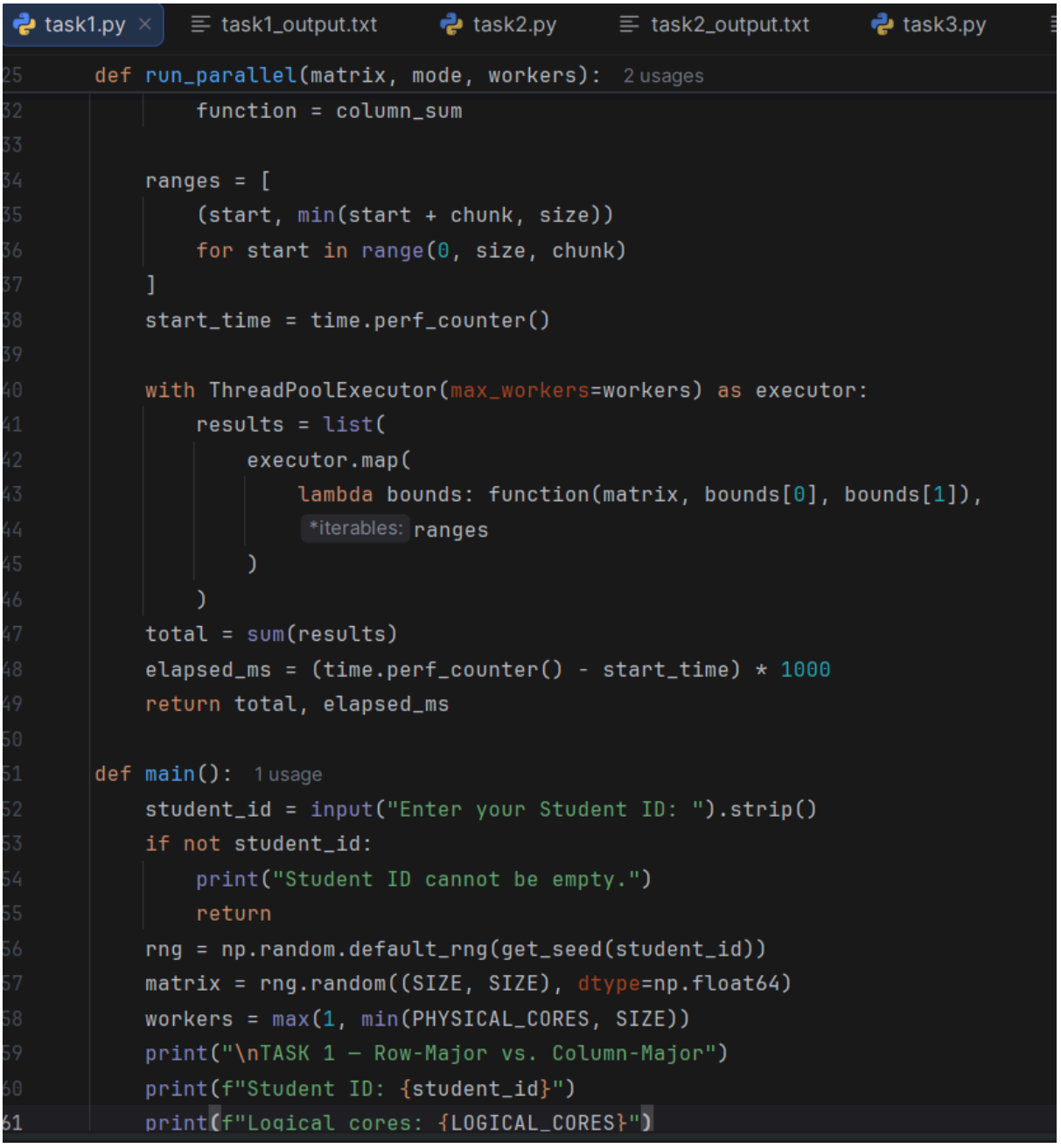

task1.py
task1_output.txt
task2.py
task2_output.txt
task3.py
task3_
import os
import time
import hashlib
import numpy as np
from concurrent.futures import ThreadPoolExecutor
try:
import psutil
PHYSICAL_CORES = psutil.cpu_count(logical=False) or os.cpu_count() or 1
except ImportError:
PHYSICAL_CORES = os.cpu_count() or 1
LOGICAL_CORES = os.cpu_count() or 1
SIZE = 8000
def get_seed(student_id): 1 usage
value = hashlib.sha256(student_id.encode()).hexdigest()
return int(value[:8], 16)
def row_sum(matrix, start, end): 1 usage
return np.sum(matrix[start:end, :])
def column_sum(matrix, start, end): 1 usage
total = np.float64(0.0)
for col in range(start, end):
total += np.sum(matrix[:, col])
return total
def run_parallel(matrix, mode, workers): 2 usages
size = matrix.shape[0]
chunk = (size + workers - 1) // workers
if mode == "row":
function = row_sum
else:


task1.py
task1_output.txt
task2.py
task2_output.txt
task3.py
def run_parallel(matrix, mode, workers): 2 usages
function = column_sum
ranges = [
(start, min(start + chunk, size))
for start in range(0, size, chunk)
]
start_time = time.perf_counter()
with ThreadPoolExecutor(max_workers=workers) as executor:
results = list(
executor.map(
lambda bounds: function(matrix, bounds[0], bounds[1]),
*iterables: ranges
)
)
total = sum(results)
elapsed_ms = (time.perf_counter() - start_time) * 1000
return total, elapsed_ms
def main(): 1 usage
student_id = input("Enter your Student ID: ").strip()
if not student_id:
print("Student ID cannot be empty.")
return
rng = np.random.default_rng(get_seed(student_id))
matrix = rng.random((SIZE, SIZE), dtype=np.float64)
workers = max(1, min(PHYSICAL_CORES, SIZE))
print("\nTASK 1 — Row-Major vs. Column-Major")
print(f"Student ID: {student_id}")
print(f"Logical cores: {LOGICAL_CORES}")

task1.py | task1_output.txt | task2.py | task2_output.txt | task3.py | task3_

```
    def main():  1 usage
        print("\nTASK 1 — Row-Major vs. Column-Major")
        print(f"Student ID: {student_id}")
        print(f"Logical cores: {LOGICAL_CORES}")
        print(f"Physical cores used: {workers}")
        print(f"Matrix size: {SIZE} x {SIZE}")
        print(f"Data type: {matrix.dtype}")

        row_result, row_time = run_parallel(matrix, mode: "row", workers)
        column_result, column_time = run_parallel(matrix, mode: "column", workers)
        penalty = column_time / row_time if row_time > 0 else float("inf")
        print(f"\nMode A (Row-Major): {row_time:.2f} ms")
        print(f"Mode B (Column-Major): {column_time:.2f} ms")
        print(f"Cache penalty factor: {penalty:.2f}x")
        print(f"Mode A sum: {row_result:.10f}")
        print(f"Mode B sum: {column_result:.10f}")
        with open("task1_output.txt", "w", encoding="utf-8") as file:
            file.write("TASK 1 — Row-Major vs. Column-Major\n")
            file.write(f"Student ID: {student_id}\n")
            file.write(f"Logical cores: {LOGICAL_CORES}\n")
            file.write(f"Physical cores used: {workers}\n")
            file.write(f"Matrix size: {SIZE} x {SIZE}\n")
            file.write(f"Data type: {matrix.dtype}\n")
            file.write(f"Mode A (Row-Major): {row_time:.2f} ms\n")
            file.write(f"Mode B (Column-Major): {column_time:.2f} ms\n")
            file.write(f"Cache penalty factor: {penalty:.2f}x\n")
            file.write(f"Mode A sum: {row_result:.10f}\n")
            file.write(f"Mode B sum: {column_result:.10f}\n")

        print("\nResults saved to task1_output.txt")
```

```
            file.write(f"Mode B (Column-Major): {column_time:.2f} ms\n")
            file.write(f"Cache penalty factor: {penalty:.2f}x\n")
            file.write(f"Mode A sum: {row_result:.10f}\n")
            file.write(f"Mode B sum: {column_result:.10f}\n")

        print("\nResults saved to task1_output.txt")


if __name__ == "__main__":
    main()
```